

RAPPORT DE PROJET

*Application Web Full-Stack React + Laravel
Plateforme de commande en ligne Pâtisserie & Cuisine*

Élément	Détail
Projet	VYN'S DELIGHTS Manager E-Commerce
Auteur	Dountsop Debret
Formation	Développement Web, Niveau Approfondi (D-CLIC)
Activité	Pâtisserie, cuisine et service événementiel à Bertoua
Stack technique	React 19 + PrimeReact · Laravel 13 · MySQL · Sanctum
Date	Juillet 2026
Lien vidéo demo	https://drive.google.com/file/d/1EBk0W5Xl0sYf6lksg26yKa9nzUVHks49/view?usp=drive_link
Lien application héberger	https://vyns.elshaseries.com/
Lien github	https://github.com/Logiboss12/vyns-delights-manage

Sommaire

Sommaire	2
1. Introduction et contexte	3
Problèmes résolus	3
Objectifs du projet	Erreur ! Signet non défini.
2. Architecture technique	3
Technologies mises en œuvre	3
Organisation du dépôt	4
Communication et proxy de développement	4
3. Modèle de données	4
Hiérarchie des catégories	4
Points de conception notables	4
4. Fonctionnalités du back-end (API REST)	5
4.1 Authentification sécurisée (Sanctum)	5
4.2 Catalogue et catégories hiérarchiques	5
4.3 Commandes et gestion du stock	5
4.4 Espace administrateur	5
4.5 Téléversement d'images	5
4.6 Événements et promotions	5
5. Fonctionnalités du front-end (SPA React)	6
5.1 Parcours client	6
5.2 Authentification côté client	7
5.3 Espace d'administration	7
5.4 Design et charte graphique	7
6. Sécurité applicative	8
7. Points d'entrée de l'API	8
Routes publiques	8
Routes authentifiées (client)	9
Routes administrateur	9
8. Difficultés rencontrées et solutions	9
Enseignements clés	Erreur ! Signet non défini.
9. Validation et tests	9
10. Environnement d'exécution	10
11. Conclusion et perspectives	10
Perspectives d'évolution	Erreur ! Signet non défini.

1. Introduction et contexte

Ce rapport présente la conception et la réalisation de l'application web « VYN'S DELIGHTS Manager E-Commerce », développée dans le cadre du cours de Développement Web de niveau approfondi. VYN'S DELIGHTS est une activité de pâtisserie et de cuisine maison basée à Bertoua, proposant également un service pour les événements du quotidien : anniversaires, mariages, surprises et commandes spéciales ou individuelles.

L'application remplace un suivi jusqu'ici manuel (cahier, appels téléphoniques, WhatsApp) par une plateforme web structurée. Elle permet aux clients de consulter le catalogue, de commander en ligne et de suivre leurs commandes, tout en offrant à la gérante un espace d'administration pour piloter son catalogue, ses stocks, ses commandes et ses événements.

Problèmes résolus

- Perte d'informations lors de la prise de commande orale.
- Absence de suivi du stock en temps réel et risque de survente.
- Manque de traçabilité des commandes et de visibilité sur l'activité.

Objectifs du projet

- **Objectif principal** : offrir un canal de vente en ligne fiable et une gestion centralisée pour la gérante.
- **Objectif pédagogique** : mettre en œuvre une architecture complète React + Laravel avec authentification sécurisée (Sanctum), CRUD, gestion des rôles et bonnes pratiques.

2. Architecture technique

L'application suit une architecture Client-Serveur découplée : une application monopage (SPA) React côté client communique avec une API REST Laravel côté serveur, au moyen de requêtes HTTP au format JSON. Cette séparation nette permet de faire évoluer indépendamment l'interface et la logique métier.

Technologies mises en œuvre

Couche	Technologie	Rôle
Front-end	React 19	Interface utilisateur (SPA)
UI	PrimeReact + PrimeIcons	Composants d'interface prêts à l'emploi
Navigation	React Router	Routage côté client
État global	Context API	Panier et utilisateur authentifié
HTTP	Axios	Appels à l'API + intercepteur de jeton
Graphiques	Chart.js	Visualisations du tableau de bord
Back-end	Laravel 13	API REST, logique métier
Base de données	MySQL 8	Stockage relationnel
Authentification	Laravel Sanctum	Jetons d'API par utilisateur
Build front	Vite	Serveur de développement et bundler

Organisation du dépôt

Le projet est structuré en mono-dépôt, séparant clairement le back-end et le front-end, ce qui facilite le versionnage avec Git et le déploiement.

Vyns -delights-manager/		
	backend/ <	API REST Laravel 13 (PHP, MySQL, Sanctum)
	frontend/ <	SPA React + PrimeReact (Vite)
	README.md	

Communication et proxy de développement

En développement, le front-end (port 5173) et l'API (port 8000) tournent sur des ports distincts. Un proxy Vite redirige les requêtes commençant par /api vers Laravel, ce qui évite les problèmes de CORS. Le jeton Sanctum est transmis automatiquement dans l'en-tête Authorization de chaque requête authentifiée, via un intercepteur Axios.

3. Modèle de données

La base repose sur six entités principales. Une particularité notable : les catégories sont hiérarchiques, ce qui permet d'organiser le catalogue en deux univers (Cuisine et Pâtisserie) comportant chacun leurs sous-catégories.

Entité	Description	Relations principales
User	Utilisateur (client ou admin)	Possède plusieurs commandes
Category	Catégorie hiérarchique	Parent / sous-catégories, produits
Product	Plat ou pâtisserie	Appartient à une (sous-)catégorie
Order	Commande d'un client	Appartient à un user, a des lignes
OrderItem	Ligne de commande	Relie une commande et un produit
Event	Événement / promotion	Autonome (géré par l'admin)

Hiérarchie des catégories

La table categories possède une colonne parent_id qui pointe vers la même table (relation réflexive). Les catégories racines (parent_id null) sont les deux univers ; leurs enfants sont les sous-catégories auxquelles les produits sont réellement rattachés.

CUISINE	PATISSERIE
Plats principaux Entrees & accomp. Grillades & braises Boissons	Gâteaux Viennoiseries Entremets & desserts Gâteaux événementiels

4. Fonctionnalités du back-end (API REST)

4.1 Authentification sécurisée (Sanctum)

Inscription, connexion et déconnexion via des jetons d'API Laravel Sanctum. À l'inscription, la validation est déléguée à un Form Request, le mot de passe est haché automatiquement, et le rôle est forcé à « client ». Un middleware personnalisé « is_admin » protège l'ensemble de l'espace administrateur.

4.2 Catalogue et catégories hiérarchiques

- **Consultation publique** : liste paginée des produits (10 par page), filtrable par catégorie, et fiche détaillée.
- **Arborescence** : une route dédiée renvoie les deux univers avec leurs sous-catégories imbriquées, pour alimenter le filtre du catalogue.
- **CRUD admin** : création, modification et suppression de produits et de catégories, réservées à l'administratrice.

4.3 Commandes et gestion du stock

La création de commande est la logique la plus sensible. Elle est encapsulée dans une transaction de base de données garantissant le principe « tout ou rien » : vérification du stock, décrémentation, calcul du total et création des lignes se font de manière atomique.

- **Vérification du stock** : commande refusée (422) si la quantité dépasse le disponible.
- **Verrou de concurrence** : lockForUpdate prévient la survente en cas de commandes simultanées.
- **Calcul serveur** : le montant est toujours calculé côté serveur, jamais d'après le client.

4.4 Espace administrateur

- **Tableau de bord** : statistiques (commandes en attente, totaux) et alerte des produits sous le seuil de stock (≤ 5).
- **Statistiques graphiques** : top des produits vendus, répartition des commandes par statut, chiffre d'affaires sur 7 jours.
- **Commandes** : vue de toutes les commandes et mise à jour de leur statut

4.5 Téléversement d'images

Une route protégée permet à l'administratrice de téléverser une photo depuis son ordinateur. Le fichier est validé (type image, formats jpeg/png/webp, taille ≤ 2 Mo), stocké dans storage/app/public/products et rendu accessible via un lien symbolique. L'URL renvoyée alimente le champ image_url du produit.

4.6 Événements et promotions

Un module CRUD complet permet de gérer des événements (promotions, prestations, nouveautés) affichés dynamiquement sur la page d'accueil. Chaque événement possède un type, un badge, un libellé de date et un indicateur d'activation permettant de le masquer sans le supprimer.

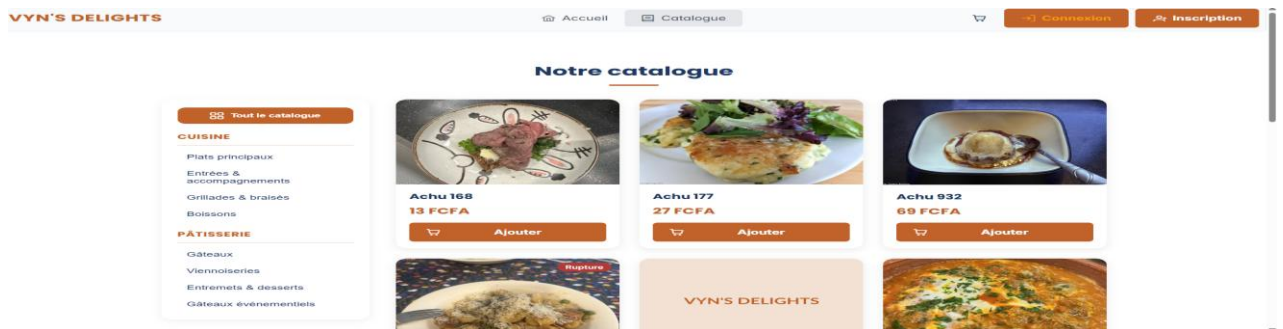
5. Fonctionnalités du front-end (SPA React)

L'interface s'appuie sur la bibliothèque de composants PrimeReact pour offrir une expérience soignée et cohérente, dans le respect de la charte graphique de VYN'S DELIGHTS.

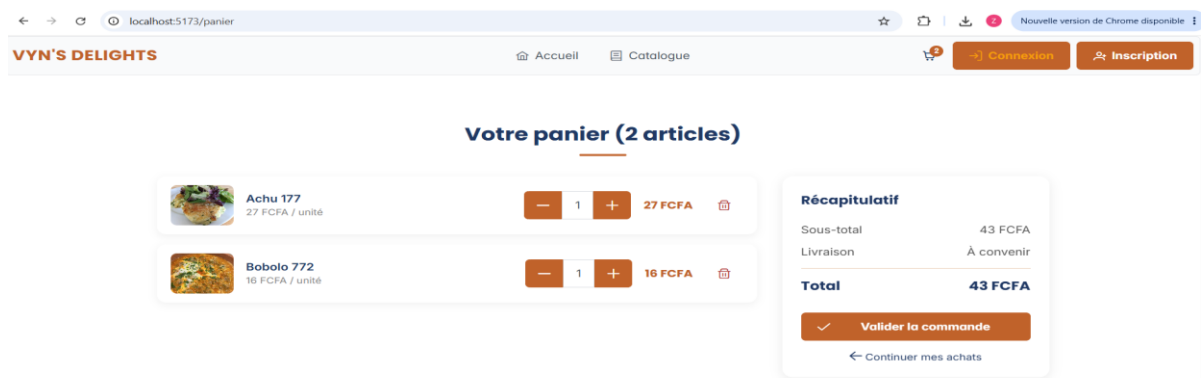
5.1 Parcours client



- Page d'accueil de l'application



- Catalogue avec filtre



- Panier avec articles

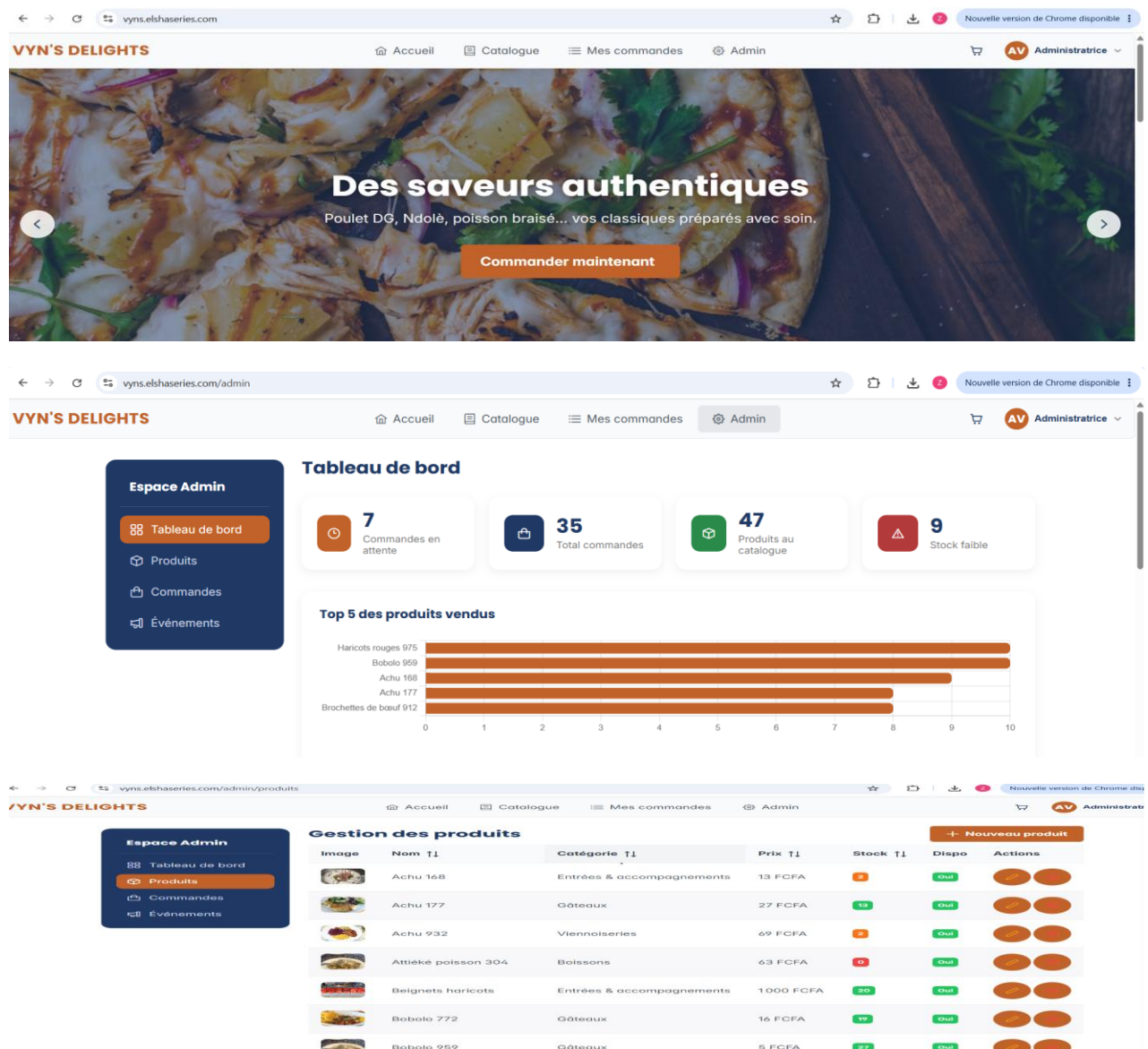


- Mes commandes

5.2 Authentification côté client

Un contexte d'authentification conserve l'utilisateur connecté et son jeton. L'en-tête s'adapte à l'état de connexion : avatar avec menu déroulant, accès « Mes commandes » et espace admin pour la gérante. Les routes sensibles sont protégées par un composant de garde qui redirige les visiteurs non autorisés.

5.3 Espace d'administration





5.4 Design et charte graphique

La charte repose sur trois couleurs : le terracotta (C0622A) comme couleur principale évoquant le fait-maison, le bleu marine (1F3864) pour les titres et l'espace admin, et un fond crème (F3E1D3) pour les sections mises en avant. Les titres utilisent Poppins, le texte courant Inter. Les badges de statut de commande suivent un code couleur cohérent : gris (en attente), bleu (confirmée), orange (en préparation), vert (livrée), rouge (annulée).

6. Sécurité applicative

La sécurité, point central du cahier des charges, repose sur plusieurs mécanismes complémentaires conformes aux bonnes pratiques Laravel et React.

Menace	Protection mise en place
Accès non autorisé	Middleware auth:sanctum + is_admin sur les routes sensibles
Injection SQL	ORM Eloquent (requêtes paramétrées)
Mots de passe	Hachage automatique bcrypt (cast « hashed »)
Données invalides	Validation stricte côté serveur (Form Requests)
Élévation de privilège	Rôle admin non attribuable à l'inscription
Faible XSS	Échappement automatique de React
Survente / concurrence	Transactions + verrou lockForUpdate
Fichiers malveillants	Validation type et taille à l'upload d'image

7. Points d'entrée de l'API

Routes publiques

Méthode	Route	Fonction
POST	/api/register	Inscription d'un client
POST	/api/login	Connexion
GET	/api/category-tree	Arborescence des catégories
GET	/api/products	Liste paginée des produits
GET	/api/products/{id}	Détail d'un produit
GET	/api/events	Événements actifs

Routes authentifiées (client)

Méthode	Route	Fonction
GET	/api/orders	Historique des commandes
POST	/api/orders	Passer une commande
POST	/api/logout	Déconnexion
GET	/api/user	Profil connecté

Routes administrateur

Méthode	Route	Fonction
GET	/api/admin/dashboard/stats	Statistiques
GET	/api/admin/dashboard/charts	Données des graphiques
GET	/api/admin/orders	Toutes les commandes
PATCH	/api/admin/orders/{id}/status	Changer le statut
POST	/api/products · /api/events	Créer produit / événement
POST	/api/upload/image	Téléverser une image

8. Difficultés rencontrées et solutions

Le développement a comporté plusieurs incidents dont la résolution constitue un apprentissage précieux. Ils sont documentés ci-dessous.

Difficulté	Cause	Solution
Clé d'index trop longue	utf8mb4 + VARCHAR(255)	Schema::defaultStringLength(191)
OverflowException (Faker)	unique() sur liste trop courte	Retrait de unique()
\$request->user() null	Route hors du groupe auth:sanctum	Réintégration dans le bon groupe
Route /tree en 404	Conflit avec /{category}	URL distincte /category-tree
Route en 500 « login »	Route placée sous is_admin	Déplacement en zone publique
Erreur React <Route>	Balises mal imbriquées	Réécriture propre de App.jsx
Bouton supprimer coloré	Règle CSS trop large	Sélecteur :not() ciblé

9. Validation et tests

Chaque fonctionnalité a été vérifiée manuellement, côté API (requêtes HTTP) et côté interface (parcours réels). Le tableau ci-dessous synthétise les principaux scénarios validés.

Scénario testé	Résultat attendu	Statut
Inscription / connexion (Sanctum)	Jeton délivré	Validé
Catalogue paginé + filtre par univers	Produits filtrés	Validé
Passage de commande	Commande + décrétement stock	Validé
Stock insuffisant	Erreur 422	Validé
Suivi « Mes commandes »	Statuts affichés	Validé

CRUD produits + upload image	Produit avec photo	Validé
Gestion des statuts (admin)	Statut mis à jour	Validé
Événements dynamiques	Affichés sur l'accueil	Validé
Graphiques du dashboard	Données agrégées	Validé
Accès admin par un client	Erreur 403	Validé

Des données de démonstration cohérentes (catalogue hiérarchique, événements, commandes réparties dans le temps et par statut) sont injectées par des seeders, ce qui rend l'application immédiatement présentable et les graphiques significatifs.

10. Environnement d'exécution

En démonstration, l'application s'exécute en local : le serveur Laravel (Artisan Serve, port 8000) fournit l'API, et le serveur de développement Vite (port 5173) sert la SPA React. Les prérequis sont PHP 8.5, Composer, Node.js, MySQL et Git.

11. Conclusion et perspectives

L'application VYN'S DELIGHTS est fonctionnellement complète : une vitrine soignée présentant les deux univers (pâtisserie et cuisine) et les services événementiels, un parcours client de bout en bout (du catalogue au suivi de commande), et un espace d'administration riche (catalogue, stocks, commandes, événements, statistiques graphiques). Le tout repose sur une API REST sécurisée et une base de données cohérente.

L'architecture découplée et modulaire constitue une base solide et évolutive, permettant d'ajouter aisément de nouvelles fonctionnalités sans remettre en cause l'existant.

Perspectives d'évolution

- **Mot de passe oublié** : réinitialisation par email (fonctionnalité en cours d'intégration).
- **Paieement en ligne** : intégration mobile money ou carte.
- **Notifications** : alertes SMS ou push sur l'avancement des commandes.
- **Tests automatisés** : couverture Pest/PHPUnit des règles métier critiques.